

Humanoid Robot

Andrii Ozgovych

Infineon

UCU

Lviv, Ukraine

Andriy.Ozhovych@infineon.com

Mykhailo Ivasiuk

Computer Science, UCU

Ivano-Frankivsk, Ukraine

ivasiuk.pn@ucu.edu.ua

Bohdan Milian

Computer Science, UCU

Lviv, Ukraine

milian.pn@ucu.edu.ua

Vladyslav Sydorak

Computer Science, UCU

Lviv, Ukraine

sydorak.pn@ucu.edu.ua

Oleh Shtohryn

Computer Science, UCU

Ivano-Frankivsk, Ukraine

shtogryn.pn@ucu.edu.ua

Abstract—This document serves as a guide and model for developing a robot control system using servo motors, and custom behavior algorithms. The focus of this work is on hardware integration and the development of a script-based control system that plays behaviors from stored files. The project emphasizes the manipulation of a metal-framed robot with embedded hardware components such as the PCA9685 and ESP32-S3 modules

I. INTRODUCTION

This document provides a framework for the implementation of a robotic control system, focusing on preparatory work to establish servo motor control. Additionally, it covers the development and integration of multiple hardware components, including a PCA9685 board and an ESP32-S3 - module, inside the robot. The primary objective is to develop a script that enables the robot to execute pre-programmed movements based on stored behavior files.

II. METHODS

A. ESP32

ESP32 is a series of low-cost, low-power system-on-chip microcontrollers. ESP32-S3 was chosen based on the recommendation of the mentor because of its affordability and relative simplicity in development, while at the same time it is a more professional and flexible platform than Arduino. The ESP32-S3 has a low-power Xtensa dual-core 32-bit LX7 microprocessor. It has an operating temperature ranging from -40°C to $+125^{\circ}\text{C}$. Powered by advanced calibration circuitries, engineered for mobile devices, wearable electronics and IoT¹ applications. ESP32 can interface with other systems to provide Wi-Fi and Bluetooth functionality through its SPI² / SDIO³ or I2C / UART interfaces. In this project we use both I2C and UART as communication methods (which are explained further).

¹The Internet of Things (IoT) refers to a network of physical devices, vehicles, appliances, and other physical objects that are embedded with sensors, software, and network connectivity, allowing them to collect and share data.

²Serial Peripheral Interface

³SDIO (Secure Digital Input Output) - an extension of the SD specification to cover I/O functions

B. PCA9685

PCA9685 was chosen for ease of use and affordability. With PCA9685 as the master chip, the 16-channel 12-bit PWM⁴ Servo Driver only needs 2 I2C pins to control 16 servos, thus greatly reducing the occupant I/Os. Moreover, it can be connected to 62 driver boards at most in a cascade way, which means that it will be able to control 992 servos in total. There are 16 output ports. Each port has 3 pins: V+, GND and the PWM output. Each PWM runs completely independently, but they must all have the same frequency.

Features of the PCA9685:

- Noise filter on SDA/SCL inputs
- Supports hot insertion
- Operating power supply voltage range of 2.3 V to 5.5 V
- -40°C to $+85^{\circ}\text{C}$ operation temperature

C. UART

In UART communication, two UARTs communicate directly with each other. The transmitting one converts parallel data from a controlling device like a CPU into serial form, transmits it in serial to the receiving one, which then converts the serial data back into parallel data for the receiving device. Only two wires are needed to transmit data between two UARTs. The transmitter collects data from a source, formats it into serial bits, and sends it via a TX (Transmit) pin. The receiver receives it via a RX (Receive) pin, processes incoming serial data and converts it into parallel data for the host system.

UARTs transmit data asynchronously, which means that there is no clock signal to synchronize the output of bits from the transmitting one to the sampling of bits by the receiving one. Instead of a clock signal, the transmitting UART adds start and stop bits to the data packet being transferred. These bits define the beginning and end of the data packet so the receiving UART knows when to start reading the bits.

When the receiving UART detects a start bit, it starts to read the incoming bits at a specific frequency known as the baud

⁴Pulse width modulation or PWM is a commonly used control technique that generates analog signals from digital devices such as microcontrollers. The signal thus produced will have a train of pulses, and these pulses will be in the form of square waves

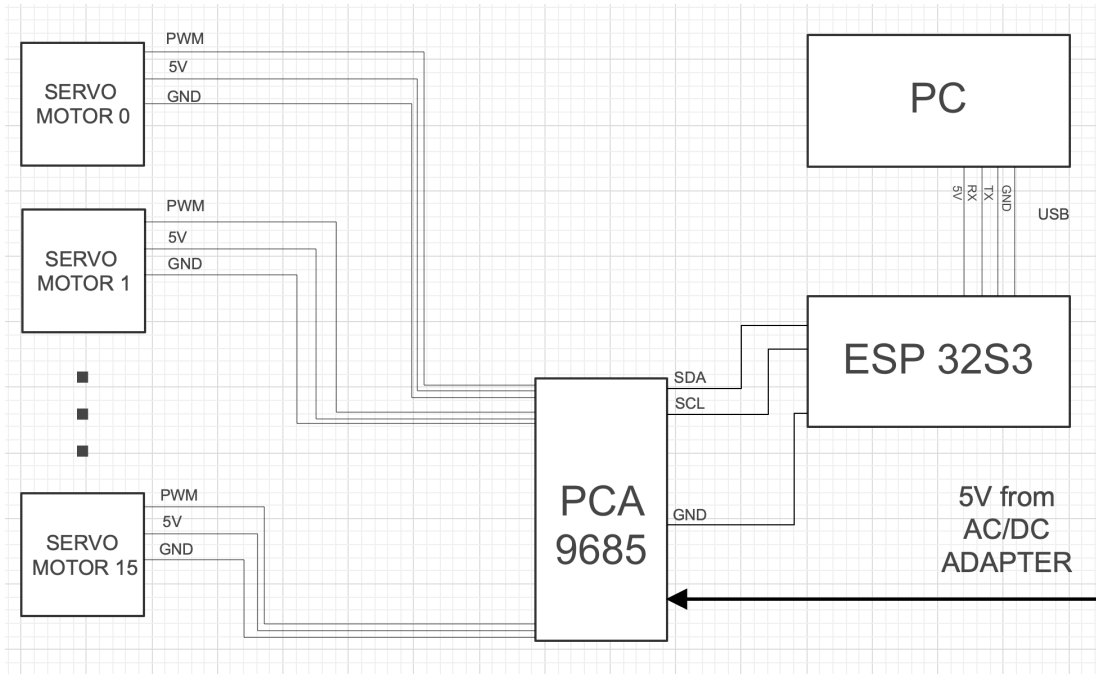


Fig. 1. Diagram of the robot hardware integration showing connections between PCA9685 and ESP32-S3.

rate. Baud rate is a measure of the speed of data transfer, we use baudrate of 19200 bits per second, which is enough for our use cases.

D. I2C

Like UART communication, I2C only uses two wires to transmit data between devices:

- SDA (Serial Data) – The line for the master and slave to send and receive data.
- SCL (Serial Clock) – The line that carries the clock signal.

I2C is a serial communication protocol, so data is transferred bit by bit along a single wire (the SDA line). It is synchronous, so the output of bits is synchronized to the sampling of bits by a clock signal shared between the master and the slave. The clock signal is always controlled by the master.

III. PROJECT DESCRIPTION

A. Preparatory Work

The initial stage involves connecting the servo motors to the all necessary hardware, and then to program direct control. Test the movement of the robot to confirm proper operation.

B. Main Line of Work

- **Hardware Integration:** Mount a PCA9685 board and an ESP32-S3 module inside the robot. This hardware will handle communication with the motors and execute movement commands.
- **String Parser:** Developing a string parser to interpret movement commands for the robot. The parser will process strings defining behavior algorithms such as walking

forward, stopping, and translate them into commands understood by the robot's control system.

- **Strings to UART signals:** Processing the received strings into signals to Servo Motors.

C. Recording and Playback

A key aspect of this project is the ability to record the robot's movements and store them in a file. The recorded behaviors will be played back by the robot using a "interpreter" that reads and executes behavior files from a flash drive, analogous to playing an MP3 file where the behavior is "recorded" and executed.

IV. IMPLEMENTATION

A. Hardware Integration

In the initial phase of our project, we connected the servo motors to the PCA9685 board, with an ESP32-S3 module serving as the communication bridge. Signals were transmitted from a laptop to the ESP32-S3 module via a USB connection. The ESP32-S3 then forwarded these signals to the PCA9685 boards, which controlled the servo drivers.

At first, all components were mounted on a breadboard. However, this setup proved to be unstable due to loose connections and potential breakdowns. To enhance stability and reliability, we transitioned to a perfboard. The PCA9685 board and ESP32-S3 module were soldered onto the perfboard, ensuring secure and durable connections (Fig 1).

The redesigned build was then installed inside the robot. This setup significantly improved the stability of the control system.

A big metal frame with wheels holds robot by hooks built on its chest.

B. String Parser

A robotic industry Hiwonder⁵ has a similar biped robot, that uses its own software for control. The program has a visual interface for controlling robot using sliders, or creating a sequence of individual servo motor movement. Also written programs can be transformed to files. The files have data, which is used by the stm32⁶. Since we use ESP32-S3 we needed something to convert those example files into readable for us code. Analyzing the hexadecimal view of those files, we found out how to convert those into readable for human sight. We used those files to write faster adequate code for our robot.

Our next step involved developing a string parser capable of interpreting commands in the format: `"#{servoNum}P{servoPosition}"` and at the end of each line we have `"#T{timeOfExecution}"` and `"#D{DelayBeforeNextLineExecution}"`. This parser processes a sequence of such strings and translates them into a format understood by our parser which uses our custom ServoMotors class written in C++. The parser then uses these commands to move the servo motors to the desired positions specified in the input strings.

C. Strings to UART signals

This function was implemented to test recorded strings without delays. The user connects the ESP32 to a computer via USB and runs the program. It reads the tapes line by line and sends commands to the ESP32 via UART using the "Serial" library. (The scheme of parsing strings is depicted on fig. 2).

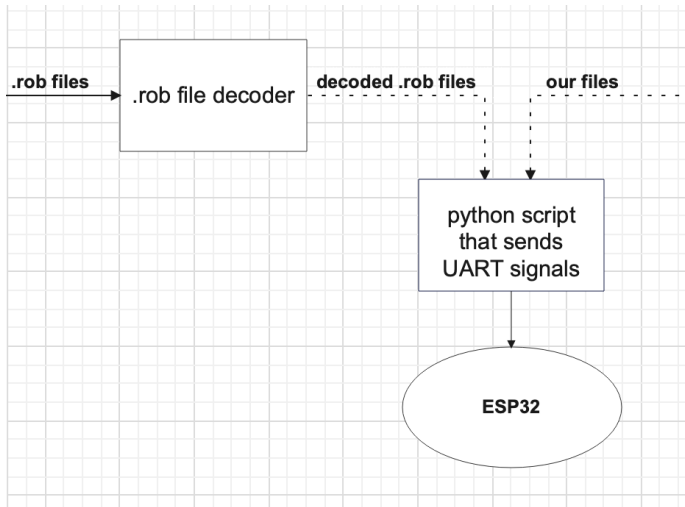


Fig. 2. Strings to UART signals

D. Interpreter on native SDK

Initially we used Arduino framework for ESP32-S3 to communicate with PCA9685. Due to the requirements of the project, everything was rewritten using native Espressif SDK.

E. Work with SD card

SD card is connected to ESP32 via built-in SDMMC (Secure Digital Memory Card Controller). Basic use case of SD card is: writing commands to some textfile, then processing the file using the interpreter.

F. The problems with the mathematical approach

The initial goal of algorithm appliance was to use mathematical description of its movement. We spent a lot of time in the process of figuring out what to do, we came to the conclusion that it is too complicated for us. In order to do what we wanted to, we would need too much time.

Implementing a walking algorithm for a 17-DOF (Degree of Freedom⁷) humanoid robot is challenging due to the need of precise coordination of multiple joints while maintaining dynamic stability and balance with ZMP (Zero Moment Point⁸) control. It requires integration of inverse kinematics⁹ constrained by the physical limits of servo motors. Thus, achieving human-like gait further adds complexity, making this a time-consuming and high-profile problem.

G. The implementation of the walking algorithm

Our actual implementation of walking algorithm relies on live modeling of robot movements while using the supporting metal frame in order to save the subject from falling during testing.

H. Concrete stages of our robot walking

- **Stage "A"** The robot is standing firmly on the ground
- **Stage "B"** The robot is shifting its weight to the left and sticks out his right leg in order to then place it on the ground.
- **Stage "C"** The robot moves his hips and proceeds to shift its supporting leg back to be in balance and advances even further with his leading leg.
- **Stage "D"** The robot completely places its leg on the ground with the help of his supporting and leading legs.
- **Stage "E"** The robot shifts its weight to the right and raises its leading leg.
- **Stage "F"** The robot advances with the leading leg while moving the supporting leg to stabilize itself. The hips are moved for additional center-of-mass management.
- **Stage "G"** The robot again hummed the weight on the opposite leg and stuck his other limb out.
- **Stage "H"** The process is continued from the stage "B" and looped for the other leg.

⁷DOF (Degree of Freedom) - the number of movable joints of motion mounted on the robot.

⁸ZMP - the point with respect to which reaction forces at the contacts between the feet and the ground do not produce any moment in the horizontal direction. The concept assumes the contact area is planar and has sufficiently high friction to keep the feet from sliding.

⁹Inverse kinematics - the use of kinematic equations to determine the motion of a robot to reach a desired position.

⁵Hiwonder - a company that manufactures robots

⁶STM32 is a family of 32-bit microcontroller integrated circuits by STMicroelectronics.

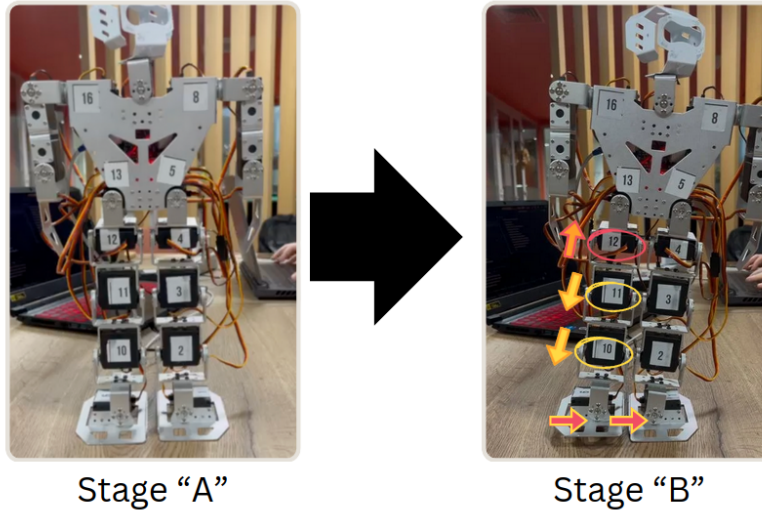


Fig. 3. Robot walking stages 1

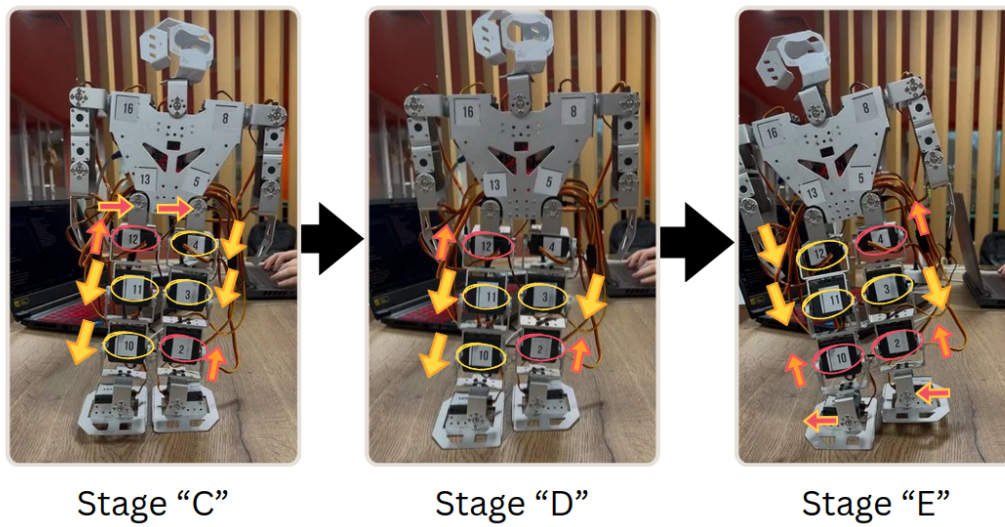


Fig. 4. Robot walking stages 2

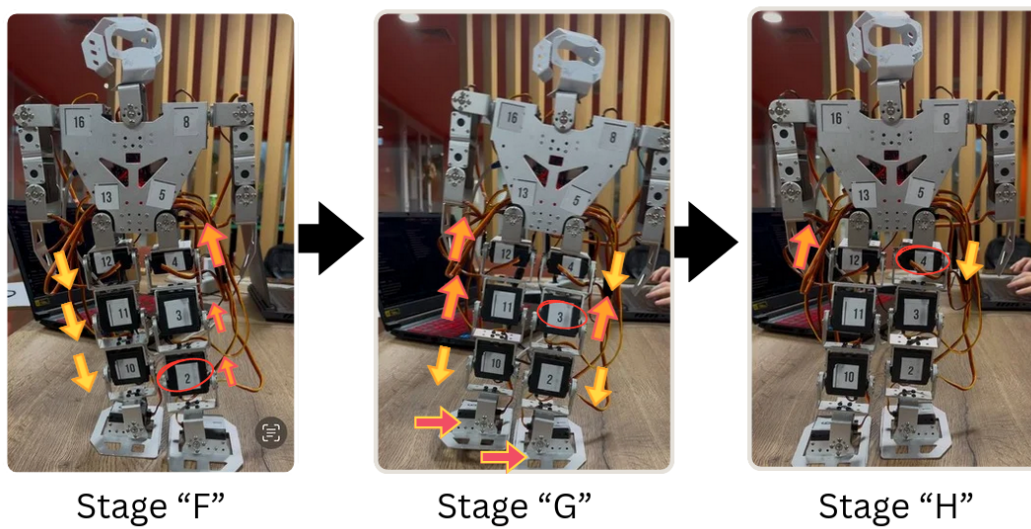


Fig. 5. Robot walking stages 3

THE PROBLEMS WE ENCOUNTERED DURING OUR WORK

I. The inadequate work of PCA9685

Through our work we encountered the issue working with PCA9685. Every piece of this hardware might have slight deviations in PWM signal it conveys. With the work so delicate, like programming precise movements of the robot, those variations are vital.

J. Weakening of the servos during intense sessions

Recently, servo motors have become very weak when we run our robot. This has caused significant problems, as they have stopped forging complex movements with weight transfer.

K. The problem with power

The AC/DC adapter outputs only 5V when the servo motors support 6.8V, meaning that not all of their potential is used.

V. FUTURE

Some ideas to develop the project in the future:

1. Replace the AC/DC adapter to unleash the full potential of servomotors.
2. Achieve the Energy Independence.
3. Add sensors to implement self-balancing.
4. Add remote control via built-in Bluetooth in the ESP32.

REFERENCES

- [1] How-to: 17 DOF Humanoid Robot By IgorF2 in CircuitsRobots. <https://www.instructables.com/How-to-17-DOF-Humanoid-Robot/>.
- [2] Embedded Walking Algorithm for Biped Humanoid Robot with 17 Degrees-of-Freedom. Available: <https://iopscience.iop.org/article/10.1088/1757-899X/618/1/012004/pdf>.
- [3] Link to the official documentation of ESP32S3 https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [4] Link to the official documentation of PCA9685 <https://cdn-shop.adafruit.com/datasheets/PCA9685.pdf>
- [5] Basic principles of uart protocol <https://www.circuitbasics.com/basics-uart-communication/>
- [6] Basic principles of I2C protocol <https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/>
- [7] Hiwonder company <https://www.hiwonder.com/?srsltid=AfmBOoqF7S-1bP2h6Iw8hJbwJ0IHcsZ-W9CiHELwM1n0K6Uwr7fdkPwn>
- [8] IDE which we use for our project <https://docs.espressif.com/projects/espressif-ide/en/latest/index.html>